

Project CCA

Computational Optimizations of Structured Matrices, Toeplitz case

Ali Arda BARUT
Rafael LLAMAS

April 17, 2026

Contents

1	Introduction	2
2	Definitions	2
2.1	Toeplitz Matrices	2
2.1.1	Quasi Toeplitz Matrices	2
2.2	Displacement Matrices	2
2.2.1	Mathematical Approach	3
2.2.2	Computational Approach	3
2.3	Generator Matrices	3
2.4	Displacement Operators	4
2.5	Conclusion & Implementation	4
3	Operation	4
3.1	Addition	4
3.1.1	Introduction	5
3.1.2	Exemple	5
3.2	Multiplication	6
3.2.1	Introduction	6
3.2.2	Multiplication Vecteur/Générateur	6
3.2.3	Exemple	8
3.3	Generator Compression	9
3.4	Inversion	10
3.4.1	Introduction	10
3.4.2	Block Decomposition of the Displacement Operator	11
3.4.3	Extraction of Block Generators	12

1 Introduction

Structured matrices can allow optimizations in certain matrix operations. We will be studying the cases of Toeplitz matrices and its property of diagonally descending elements.

Each section is an implementation topic on which we state our implementation, complexity and the results.

2 Definitions

We will discuss what are Toeplitz Matrices, how are they defined and what are their properties such that we can take advantage during matrix implementations.

2.1 Toeplitz Matrices

A Toeplitz matrix is in which each element in first row and first column descends diagonally. For a matrix $A \in \mathbb{K}^{n \times m}$, it satisfies the condition $A_{i,j} = A_{i+1,j+1}$. Formally, it is defined by a sequence of elements $\{a_k\}$ such that:

$$A = \begin{pmatrix} a_0 & a_{-1} & a_{-2} & \cdots & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & \cdots & a_{-(m-2)} \\ a_2 & a_1 & a_0 & \cdots & a_{-(m-3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \cdots & a_{n-m} \end{pmatrix} \quad (1)$$

$$A_{i,j} = a_{i-j}, \quad 0 \leq i < n, \quad 0 \leq j < m$$

2.1.1 Quasi Toeplitz Matrices

A Quasi-Toeplitz matrix is not strictly Toeplitz but shares similar structural properties. We can also define Quasi-Toeplitz matrix as a Toeplitz Matrix that has noise within. Notice that we can still benefit the structural advantages of Toeplitz partially in these matrices.

2.2 Displacement Matrices

For a matrix $A \in \mathbb{K}^{n \times m}$, we define the displacement matrix ∇A using:

$$\nabla A = A - ZAZ^T \quad (2)$$

where Z represents the lower shift matrix and Z^T represents the upper shift matrix. Essentially, Z is a matrix with ones on the first sub-diagonal and zeros everywhere else:

$$Z = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix} \quad Z^T = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix} \quad (3)$$

Multiplying a matrix A by Z from the left (ZA) shifts the rows of A down by one (zeros at the top). Multiplying by Z^T from the right (AZ^T) shifts the columns to the right, (zeros at the first column). Resulting in an operation ZAZ^T that shifts the entire matrix A one step to the bottom-right.

2.2.1 Mathematical Approach

For a generic Toeplitz matrix of size $n \times m$, this subtraction results in zeros everywhere except for the first row and the first column.

$$\nabla A = \begin{pmatrix} y_0 & x_1 & x_2 & \cdots & x_{m-1} \\ y_1 & 0 & 0 & \cdots & 0 \\ y_2 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{n-1} & 0 & 0 & \cdots & 0 \end{pmatrix} \in \mathbb{K}^{n \times m} \quad (4)$$

This allows us to represent the $n \times m$ matrix using only $O(n + m)$ elements rather than storing the full $O(nm)$ dense matrix.

2.2.2 Computational Approach

While mathematical definition involves matrix multiplication, in this case $O(n^2 * m)$ or similar depending on the multiplication method used, calculating ∇A based on matrix multiplication is inefficient.

The displacement calculation can be reduced to an element based loop with complexity $O(nm)$:

$$(\nabla A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = 0 \text{ or } j = 0 \\ A_{i,j} - A_{i-1,j-1} & \text{otherwise} \end{cases} \quad (5)$$

This approach allows us to iterate from the bottom row to the top updating values in place.

2.3 Generator Matrices

Displacement matrices, ∇A , typically have a low rank α , we call this displacement rank. For Toeplitz matrices, α is generally small ($\alpha \leq 2$). This allows us to store efficiently the displacement matrix into two smaller matrices G and H .

While the product $G \times H^T$ results in the displacement ∇A , the original matrix A can be reconstructed using the columns of the generators mentioned at Proposition 10.5 in [1]:

$$A - ZAZ^T = GH^T \quad \text{iff} \quad A = \sum_{i=1}^{\alpha} L(x_i)U(y_i), \quad (6)$$

where x_i (and thus y_i) are the columns of the generator G (and thus H), where for a column vector $v = [v_0 \dots v_{n-1}]^T$, $L(v)$ denotes the lower triangular Toeplitz matrix:

$$L(v) = \begin{pmatrix} v_0 & 0 & \dots & 0 \\ v_1 & v_0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & \dots & v_1 & v_0 \end{pmatrix}$$

and $U(v)$ denotes the upper triangular Toeplitz matrix $L(v)^T$.

The matrix A can be written as a sum of elements of lower and upper triangular matrices:

$$A = \sum_{k=1}^{\alpha} L(g_k)U(h_k) \quad (7)$$

- $L(g_k)$ is a lower triangular Toeplitz matrix with the first column g_k .
- $U(h_k)$ is an upper triangular Toeplitz matrix with the first row h_k^T .

2.4 Displacement Operators

We define two displacements operators:

$$\begin{aligned}\phi_+(A) &= A - ZAZ^T = \nabla A \\ \phi_-(A) &= A - Z^T AZ\end{aligned}\tag{8}$$

In other words, $\phi_-(A)$ is the subtraction of minuend A where subtrahend is the matrix A but with the columns shifted up and rows to right once.

Computationally, it follows the same path as the calculation of ∇A . Main difference being we can run this time in increasing order:

$$\phi_-(A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = n - 1 \text{ or } j = m - 1 \\ A_{i,j} - A_{i+1,j+1} & \text{otherwise} \end{cases}\tag{9}$$

Where n and m are respectfully the number of rows and columns of matrix A .

2.5 Conclusion & Implementation

We conclude the section of the definitions. Implementations of these concepts can be found in:

```
Project
\   src
|   \   displacement_matrices
|   |   displacement_matrices.c
|   |   displacement_matrices.h
```

gr_mat_displacement(gr_mat_t D, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function taking as parameter the matrix A and returns into the matrix D , indicating destination, the displacement matrix of A . According to the type of displacement we pass via the parameter **type**, it calculates the implementations (5) or (9).

int gr_mat_G_H(gr_mat_t G, gr_mat_t H, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function that computes the generator matrices G and H^T for an input matrix A . Based on the displacement equations discussed previously, the equation (7), the function first calculates the displacement matrix according to the specified type of displacement. Since the displacement of a structured matrix (like a Toeplitz matrix) inherently results in a matrix with a low rank (α), D can be factored into two smaller matrices such that $D = GH^T$. The function achieves this by performing an LU decomposition on the displacement matrix D . It then extracts the independent columns to form G and the corresponding rows to form H^T . This is where the storage economy for structured matrices come from.

int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx)

3 Operation

3.1 Addition

This code can be found in:

```

Project
\   src
|   \   operations
|   |   |   addition.c
|   |   |   addition.h

```

3.1.1 Introduction

Grace au générateurs, nous pouvons additionner nos matrices sans utiliser la formule standard. En effet, selon la propriété 10.7 et 10.8 du cours (ref cours ICI), on apprend que la somme de 2 matrices toeplitz peut être simplifiée par la concaténation de leurs générateurs de déplacement. Cela permet de passer à un coût de $O(n \times m)$ à un coût de $O(\alpha \times m)$ pour 2 matrices A et B de tailles $n \times m$ et leurs générateurs de taille α pour ceux de A et β pour ceux de B. Avec flint, on peut utiliser la fonction

```
int gr_mat_concat_horizontal(gr_mat_t RES, gr_mat_t A, gr_mat_t B, gr_ctx_t ctx);
```

pour faire la concaténation de nos générateurs.

3.1.2 Exemple

On va prendre les matrices A, B et C le résultat de l'addition de A et B. Toutes les matrices sont dans $\mathbb{Z}/_{13}\mathbb{Z}$ pour cet exemple.

$$A = \begin{pmatrix} 0 & 7 & 10 \\ 5 & 0 & 7 \\ 0 & 5 & 0 \end{pmatrix}, B = \begin{pmatrix} 9 & 12 & 4 \\ 0 & 9 & 12 \\ 3 & 0 & 9 \end{pmatrix}, C = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \quad (10)$$

si on prends les générateurs de A = (T,U) et B = (G,H)

$$T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, U = \begin{pmatrix} 5 & 0 \\ 0 & 7 \\ 0 & 10 \end{pmatrix} \quad (11)$$

$$G = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 9 & 1 \end{pmatrix}, H = \begin{pmatrix} 9 & 0 \\ 12 & 9 \\ 4 & 3 \end{pmatrix} \quad (12)$$

et que l'on fait la concaténation on obtiens

$$RES_G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 9 & 1 \end{pmatrix} \quad (13)$$

$$RES_H = \begin{pmatrix} 5 & 0 & 9 & 0 \\ 0 & 7 & 12 & 9 \\ 0 & 10 & 4 & 3 \end{pmatrix} \quad (14)$$

Dans notre projet, il suffira d'appeler la fonction:

```
int gr_mat_addition_generateur(gr_mat_t G_a, gr_mat_t H_a, gr_mat_t G_b, gr_mat_t H_b,
                               gr_mat_t G_c, gr_mat_t H_c, gr_ctx_t ctx);
```

avec les bons paramètres pour obtenir les générateurs de déplacement G et H de D.
pour vérifier notre résultat on peut utiliser la fonction

```
int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx);
```

pour recomposer la matrice que l'on a obtenue D :

$$D = \begin{pmatrix} 9 & 6 & 1 \\ 5 & 9 & 6 \\ 3 & 5 & 9 \end{pmatrix} \quad (15)$$

On remarque bien que celle-ci est identique a la matrices C et donc que pour faire l'addition de 2 matrice toeplitz, on peut utiliser seulement leurs générateurs.

3.2 Multiplication

This code can be found in:

```
Project
\   src
|   \   operations
|   |   multiplication.c
|   |   multiplication.h
```

3.2.1 Introduction

Toujours avec les générateurs, on peut simplifier la multiplication de 2 matrices et donc réduire son cout en concaténant :

- les générateurs de $A=(T,U)$ et $B=(G,H)$
- $W = Z \times A \times Z^t \times G$
- $V = B^t \times U$
- a (resp. b) un vecteur qui est la dernière colonne de $Z \times A$ (resp. $Z^t \times B$)

Cela nous permet de passer de $O(n^3)$ pour l'algo naïf a $O(\alpha^2 \times M(n))$ pour la version que nous implémentons. ICI METTRE DU CODE AVEC QUELQUE COMMENTS !!!!

3.2.2 Multiplication Vecteur/Générateur

Pour faire notre multiplication de matrice vie leurs générateurs, nous avons besoin d'avoir une fonction qui va multiplier un vecteur avec des générateurs pour éviter de recomposer les matrices.

```
int gr_mat_mul_vector(gr_mat_t Res, gr_mat_t G, gr_mat_t H, gr_mat_t X, gr_ctx_t ctx) {
    int error = GR_SUCCESS;
    slong n = gr_mat_nrows(G, ctx), m = gr_mat_nrows(H, ctx);
    slong alpha = gr_mat_ncols(G, ctx), m_cols = gr_mat_ncols(X, ctx);

    gr_poly_t pg, ph, px, p_tmp;
    gr_poly_init(pg, ctx);
    gr_poly_init(ph, ctx);
    gr_poly_init(px, ctx);
    gr_poly_init(p_tmp, ctx);

    error = gr_mat_zero(Res, ctx);
    if (error) {
        gr_poly_clear(pg, ctx);
        gr_poly_clear(ph, ctx);
        gr_poly_clear(px, ctx);
    }
```

```

    gr_poly_clear(p_tmp, ctx);
}

for (slong j = 0; j < m_cols; j++) {
    gr_poly_fit_length(px, m, ctx);
    for (slong r = 0; r < m; r++) {
        error = gr_set(gr_poly_coeff_ptr(px, r, ctx), gr_mat_entry_srcptr(X, r, j, ctx), ctx);
        if (error) {
            gr_poly_clear(pg, ctx);
            gr_poly_clear(ph, ctx);
            gr_poly_clear(px, ctx);
            gr_poly_clear(p_tmp, ctx);
        }
    }
    _gr_poly_set_length(px, m, ctx);

    for (slong k = 0; k < alpha; k++) {
        gr_poly_fit_length(pg, n, ctx);
        for (slong r = 0; r < n; r++) {
            error = gr_set(gr_poly_coeff_ptr(pg, r, ctx), gr_mat_entry_srcptr(G, r, k, ctx), ctx);
            if (error) {
                gr_poly_clear(pg, ctx);
                gr_poly_clear(ph, ctx);
                gr_poly_clear(px, ctx);
                gr_poly_clear(p_tmp, ctx);
            }
        }
        _gr_poly_set_length(pg, n, ctx);

        gr_poly_fit_length(ph, m, ctx);
        for (slong r = 0; r < m; r++) {
            error = gr_set(gr_poly_coeff_ptr(ph, r, ctx), gr_mat_entry_srcptr(H, r, k, ctx), ctx);
            if (error) {
                gr_poly_clear(pg, ctx);
                gr_poly_clear(ph, ctx);
                gr_poly_clear(px, ctx);
                gr_poly_clear(p_tmp, ctx);
            }
        }
        _gr_poly_set_length(ph, m, ctx);

        error = gr_poly_reverse(ph, ph, m, ctx);
        if (error) {
            gr_poly_clear(pg, ctx);
            gr_poly_clear(ph, ctx);
            gr_poly_clear(px, ctx);
            gr_poly_clear(p_tmp, ctx);
        }
        error = gr_poly_mul(p_tmp, ph, px, ctx);
        if (error) {
            gr_poly_clear(pg, ctx);
            gr_poly_clear(ph, ctx);

```

```

    gr_poly_clear(px, ctx);
    gr_poly_clear(p_tmp, ctx);
}

error = gr_poly_shift_right(p_tmp, p_tmp, m - 1, ctx);
if (error) {
    gr_poly_clear(pg, ctx);
    gr_poly_clear(ph, ctx);
    gr_poly_clear(px, ctx);
    gr_poly_clear(p_tmp, ctx);
}
_gr_poly_set_length(p_tmp, FLINT_MIN(gr_poly_length(p_tmp, ctx), m), ctx);

error = gr_poly_mul(p_tmp, pg, p_tmp, ctx);
if (error) {
    gr_poly_clear(pg, ctx);
    gr_poly_clear(ph, ctx);
    gr_poly_clear(px, ctx);
    gr_poly_clear(p_tmp, ctx);
}

if (gr_poly_length(p_tmp, ctx) > n) _gr_poly_set_length(p_tmp, n, ctx);

for (slong i = 0; i < gr_poly_length(p_tmp, ctx); i++) {
    error = gr_add(gr_mat_entry_ptr(Res, i, j, ctx), gr_mat_entry_ptr(Res, i, j, ctx),
        gr_poly_coeff_srcptr(p_tmp, i, ctx), ctx);
    if (error) {
        gr_poly_clear(pg, ctx);
        gr_poly_clear(ph, ctx);
        gr_poly_clear(px, ctx);
        gr_poly_clear(p_tmp, ctx);
    }
}
}
}

gr_poly_clear(pg, ctx);
gr_poly_clear(ph, ctx);
gr_poly_clear(px, ctx);
gr_poly_clear(p_tmp, ctx);
return error;
}

```

3.2.3 Exemple

On va prendre les matrices A, B et C le résultat de la multiplication de A et B. Toutes les matrices sont dans $\mathbb{Z}/_{13}\mathbb{Z}$ pour cette exemple.

$$A = \begin{pmatrix} 10 & 5 & 8 \\ 5 & 10 & 5 \\ 3 & 5 & 10 \end{pmatrix}, B = \begin{pmatrix} 0 & 3 & 0 \\ 3 & 0 & 3 \\ 0 & 3 & 0 \end{pmatrix}, C = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \quad (16)$$

si on prends les générateurs de $A = (T, U)$ et $B = (G, H)$

$$T = \begin{pmatrix} 1 & 0 \\ 7 & 1 \\ 12 & 11 \end{pmatrix}, U = \begin{pmatrix} 10 & 0 \\ 5 & 4 \\ 8 & 9 \end{pmatrix} \quad (17)$$

$$G = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, H = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 0 \end{pmatrix} \quad (18)$$

on doit calculer W et V via leurs formule donné au début.

$$W = \begin{pmatrix} 0 & 0 \\ 10 & 0 \\ 5 & 0 \end{pmatrix}, V = \begin{pmatrix} 2 & 12 \\ 2 & 1 \\ 2 & 12 \end{pmatrix} \quad (19)$$

Puis faire la concatenation de T, W et α pour obtenir RES_G et la concatenation de V, H et β pour obtenir RES_H (il faut faire la concatenation dans l'ordre que j'ai cité les éléments).

$$RES_G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 7 & 1 & 10 & 0 & 8 \\ 12 & 11 & 5 & 0 & 5 \end{pmatrix} \quad (20)$$

$$RES_H = \begin{pmatrix} 2 & 12 & 3 & 0 & 0 \\ 2 & 1 & 0 & 3 & 0 \\ 2 & 12 & 0 & 0 & 10 \end{pmatrix} \quad (21)$$

Dans notre projet, il suffira d'appeler la fonction:

```
int gr_mat_mul_generator(gr_mat_t G_c, gr_mat_t H_c, gr_mat_t G_a,
                        gr_mat_t H_a, gr_mat_t G_b, gr_mat_t H_b,
                        gr_ctx_t ctx);
```

avec les bons parametre pour obtenir les générateur de déplacement G et H de D .

pour verifier notre résultat on peut utiliser la fonction

```
int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx);
```

pour recomposer la matrice que l'on a obtenue D :

$$D = \begin{pmatrix} 2 & 2 & 2 \\ 4 & 4 & 4 \\ 2 & 0 & 2 \end{pmatrix} \quad (22)$$

La matrice D est bien égale a C donc notre facon de multiplier les 2 matrices est correct dans ce cas.

3.3 Generator Compression

Any operation done on Toeplitz matrices will increase the rank of the generators. To maintain the complexity advantage of this structure, we must perform compressions. This will appear more and more on algorithms that require arithmetics.

For any pair of matrices $G_A, H_A \in \mathbb{K}^{n \times k}$, let r be the rank of their product $G_A H_A^T$ (where $r \leq k$). Our goal is to construct a new pair generators $G_D, H_D \in \mathbb{K}^{n \times r}$ such these conditions are met:

1. The displacement product is preserved: $G_A H_A^T = G_D H_D^T$

2. The number of columns of the matrices G_D and H_D are equal and is the rank of $G_A H_A^T$.

We will achieve this by independently compression both matrices and applying the inverse of the operations done to the other.

Algorithm 1 Generator Compression

```

1: procedure GENERATOR_COMPRESS( $G_A, H_A$ )
2:    $G_1, H_1 \leftarrow \text{GENERATOR\_COMPRESS\_AUX}(G_A, H_A)$ 
3:    $H_D, G_D \leftarrow \text{GENERATOR\_COMPRESS\_AUX}(H_1, G_1)$ 
4:   return  $G_D, H_D$ 
5: end procedure

```

Algorithm 2 Generator Compression Auxiliary

```

1: procedure GENERATOR_COMPRESS_AUX( $G, H$ )
2:    $P, L, U \leftarrow \text{LU\_Decomposition}(G^T)$   $\triangleright PG^T = LU$ 
3:    $r \leftarrow \text{Number of non-zero rows in } U$   $\triangleright \text{rank}$ 
4:   if  $r < 1$  then
5:     return Generators of a 0 matrix
6:   end if
7:    $G_D \leftarrow (U_{[0:r-1],*})^T$ 
8:    $H_D \leftarrow HP^T \times L_{*,[0:r-1]}$ 
9:   return  $G_D, H_D$ 
10: end procedure

```

Computing the LU factorization of the transposed generator $G^T \in \mathbb{K}^{r \times n}$ requires $\mathcal{O}(r^2 n)$ operations. Extracting U and applying the permutation matrix P^T to H is in $\mathcal{O}(rn)$. Computing the new right factor $H_D = HP^T \times L_{*,[0:r-1]}$ involves multiplying an $n \times k$ matrix by a $k \times r$ matrix. Using standard matrix multiplication, this takes $\mathcal{O}(r^2 n)$ operations. Our implementation is using loops to get the selected amount of rows and columns but their complexities are bounded by $\mathcal{O}(r^2 n)$.

We can generalize the complexity by revising for any matrix in $\mathbb{K}^{m \times n}$ the overall time complexity for the generator compression is $\mathcal{O}(rmn)$.

Our implementation can be found in:

```

Project
\   src
|   \   compression
|   |   |   compression.c
|   |   |   compression.h

```

`int gr_mat_generator_compress(gr_mat_t G_d, gr_mat_t H_d, gr_ctx_t ctx);` Is our function that implements the algorithms talked in this chapter.

3.4 Inversion

3.4.1 Introduction

In this section we will study how we implemented Strassen's Inversion method on generators. This code can be found in:

```

Project
\   src

```

```

| \   toepnitz_inverse
| |   |   toepnitz_inverse.c
| |   |   toepnitz_inverse.h

```

3.4.2 Block Decomposition of the Displacement Operator

To perform Strassen's inversion recursively, we must first split our $n \times n$ matrix A into four sub-blocks of size $m \times m$ (where $m = n/2$). Let A be:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Similarly, we partition its displacement generators G and H (of size $n \times \alpha$) into top and bottom halves of size $m \times \alpha$:

$$G = \begin{pmatrix} G_{top} \\ G_{bottom} \end{pmatrix}, \quad H = \begin{pmatrix} H_{top} \\ H_{bottom} \end{pmatrix}$$

As mentioned before, the ϕ_+ displacement of A is given by:

$$\phi_+(A) = A - Z_n A Z_n^T = G H^T$$

Following the displacement fomula, If we divide Z_n into four $m \times m$ blocks too, it needs the last puzzle piece to connect the shifting line from the top half of the matrix to the bottom half. (In other words, we need a way to represent the part c of matrix A)

We build this using standard basis vectors e . Let e_0 be a column vector with a 1 at the top, and e_{m-1} be a column vector with a 1 at the bottom:

$$e_0 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad e_{m-1} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}$$

When we multiply the column e_0 by the row e_{m-1}^T , we get an $m \times m$ matrix of all zeros, with a single 1 in the top-right corner. This is our piece:

$$e_0 e_{m-1}^T = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} (0 \quad \dots \quad 0 \quad 1) = \begin{pmatrix} 0 & \dots & 0 & 1 \\ 0 & \dots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & 0 \end{pmatrix}$$

Using this, we can write the full shift matrix Z_n and its transpose Z_n^T in block form:

$$Z_n = \begin{pmatrix} Z_m & 0 \\ e_0 e_{m-1}^T & Z_m \end{pmatrix}, \quad Z_n^T = \begin{pmatrix} Z_m^T & e_{m-1} e_0^T \\ 0 & Z_m^T \end{pmatrix}$$

First, we multiply Z_n by A :

$$Z_n A = \begin{pmatrix} Z_m & 0 \\ e_0 e_{m-1}^T & Z_m \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} Z_m a & Z_m b \\ e_0 e_{m-1}^T a + Z_m c & e_0 e_{m-1}^T b + Z_m d \end{pmatrix}$$

Next, Z_n^T :

$$(Z_n A) Z_n^T = \begin{pmatrix} Z_m a & Z_m b \\ e_0 e_{m-1}^T a + Z_m c & e_0 e_{m-1}^T b + Z_m d \end{pmatrix} \begin{pmatrix} Z_m^T & e_{m-1} e_0^T \\ 0 & Z_m^T \end{pmatrix}$$

Let's calculate the four resulting blocks:

- **Top-Left:** $Z_m a Z_m^T$
- **Top-Right:** $Z_m a e_{m-1} e_0^T + Z_m b Z_m^T$
- **Bottom-Left:** $(e_0 e_{m-1}^T a + Z_m c) Z_m^T = e_0 e_{m-1}^T a Z_m^T + Z_m c Z_m^T$
- **Bottom-Right:** $(e_0 e_{m-1}^T a + Z_m c) e_{m-1} e_0^T + (e_0 e_{m-1}^T b + Z_m d) Z_m^T$

3.4.3 Extraction of Block Generators

By identifying the respective sub-blocks of the global displacement equation $\phi_+(A) = A - Z_n A Z_n^T = GH^T$, we can systematically extract the generators for each quadrant of A .

a: Equating the top-left blocks yields the direct displacement of a :

$$\begin{aligned} a - Z_m a Z_m^T &= G_{top} H_{top}^T \\ \phi_+(a) &= G_{top} H_{top}^T \end{aligned}$$

Since this block is unaffected, its generators are restored:

$$G_a = G_{top}, \quad H_a = H_{top}$$

b: Equating the top-right blocks to the initial equation. We isolate the terms belonging to b on the left side:

$$\begin{aligned} b - Z_m a e_{m-1} e_0^T - Z_m b Z_m^T &= G_{top} H_{bottom}^T \\ b - Z_m b Z_m^T &= G_{top} H_{bottom}^T + Z_m a e_{m-1} e_0^T \\ \phi_+(b) &= G_{top} H_{bottom}^T + (Z_m a e_{m-1}) e_0^T \end{aligned}$$

POINTER TO RAFAEL SUM GENERATOR:

$$G_b = [G_{top} \mid Z_m a e_{m-1}], \quad H_b = [H_{bottom} \mid e_0]$$

c: We follow the exact same process, but with the transpose property $(XY)^T = Y^T X^T$ to group the vectors:

$$\begin{aligned} c - e_0 e_{m-1}^T a Z_m^T - Z_m c Z_m^T &= G_{bottom} H_{top}^T \\ c - Z_m c Z_m^T &= G_{bottom} H_{top}^T + e_0 e_{m-1}^T a Z_m^T \\ \phi_+(c) &= G_{bottom} H_{top}^T + e_0 (Z_m a^T e_{m-1})^T \end{aligned}$$

THus, the generators for c are formed by concatenation:

$$G_c = [G_{bottom} \mid e_0], \quad H_c = [H_{top} \mid Z_m a^T e_{m-1}]$$

d: The internal structure of the bottom-right block corresponds exactly to the top-left block evaluated on the bottom generators

$$d = \left(\sum_{k=1}^{\alpha} L(G_{bottom,k}) U(H_{bottom,k}) \right) + a$$

In terms of generators, matrix addition corresponds to horizontal concatenation. Given that $\phi_+(a)$ is generated by G_{top} and H_{top} , we obtain:

$$G_d = [G_{bottom} \mid G_{top}], \quad H_d = [H_{bottom} \mid H_{top}]$$

References

- [1] Alin Bostan, Frédéric Chyzak, Marc Giusti, Romain Lebreton, Grégoire Lecerf, Bruno Salvy, and Éric Schost. Algorithmes efficaces en calcul formel, August 2017. 686 pages. Édition 1.0.